

Distributed Systems

Time and Ordering



CUHK

“Time” in Distributed Systems

- Who is having the “latest” copy?
- Which transaction comes first?
 - Account = \$80
 - T1: mom withdraws \$60 in HK
 - T2: girlfriend withdraws \$70 in SZ

Time in Distributed Systems

- We need time for ordering only
 - Not exactly need the time value
- Challenge:
 - Client in JP: update(x=1)
 - JP → OR → CA → VA → EU
 - Client in EU: update(x=2)
 - EU → VA → OR → CA → JP
 - OR: x=1 → x=2
 - VA: x=2 → x=1

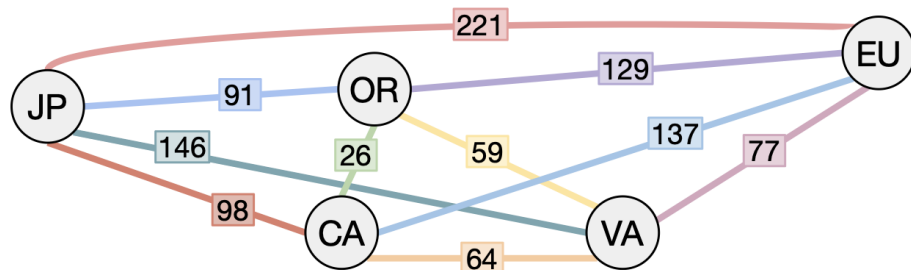


Figure 5: Round-trip times (in ms) between replicas in different locations in our test topology, as measured by the ping application. Replicas are located in Virginia (VA), California (CA), Oregon (OR), Japan (JP), and London (EU).

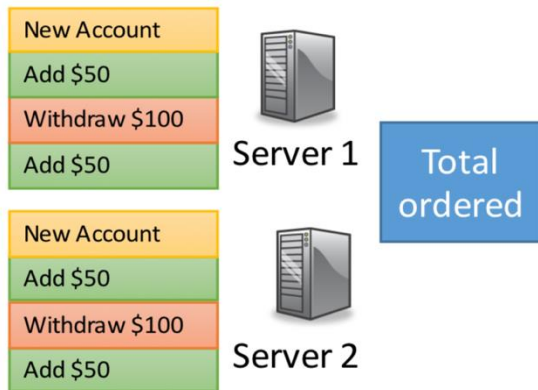
What is a “correct” order?

- **Total order:** all replicas process messages in same order
 - The **same** order among replicas, but might **not respect the real-time**
 - A.k.a. sequential consistency
- **Linearizability:** total order + **respect the real-time**
- Which one is harder to implement (achieve)?

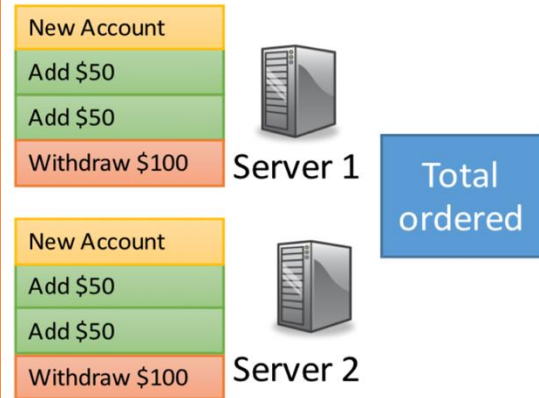
Client sends to
server 1 and server 2



Total but not linearizable



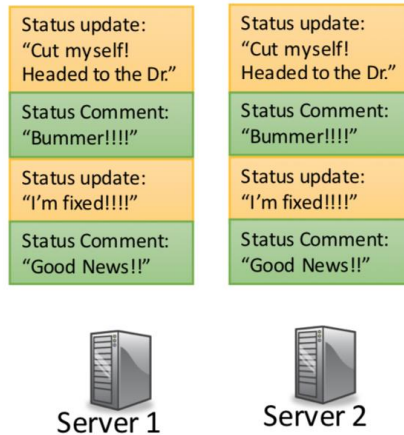
Linearizable, so also total



What is a “correct” order?

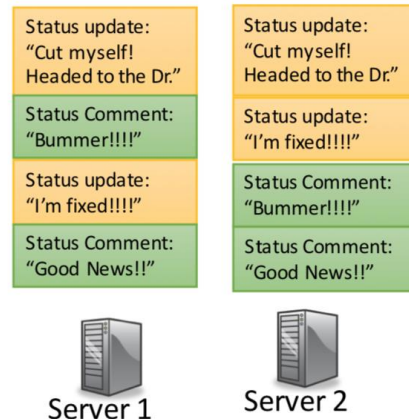
- Total order: all replicas process messages in same order
- Linearizability: total order follows real time
- **FIFO order**: different servers can get different orders as long as **the order from the same client** is preserved
 - 2 clients: yellow and green → → →
- Which order is weaker?
- Which one is easier to implement/achieve?

All servers
process in
same order



Total (!= linearizable)

Different servers
Can process in
different orders but
events from clients
in same order



FIFO: order of
each client is
preserved

What is a “correct” order?

- Total ordering is required for banking
 - All branches see the same input order and reach the same next state
- FIFO ordering is acceptable in some web applications
 - It can't tell the relationship **between processes**
- **Causal ordering** tells the causal relationship between processes
 - Those without causal relationships can get different orders in different processes, e.g.,
 - Time 1: Mary tweets A: “I lost my wedding ring!”.
 - Time 2: Mary tweets B: “Oh I found it”.
 - Time 3: Peter “reply to” tweet B: “Great”.
 - FIFO: people on Twitter see:
 - Mary: “I lost my wedding ring!”.
 - Peter: “Great”.
 - Mary: “Oh I found it”.

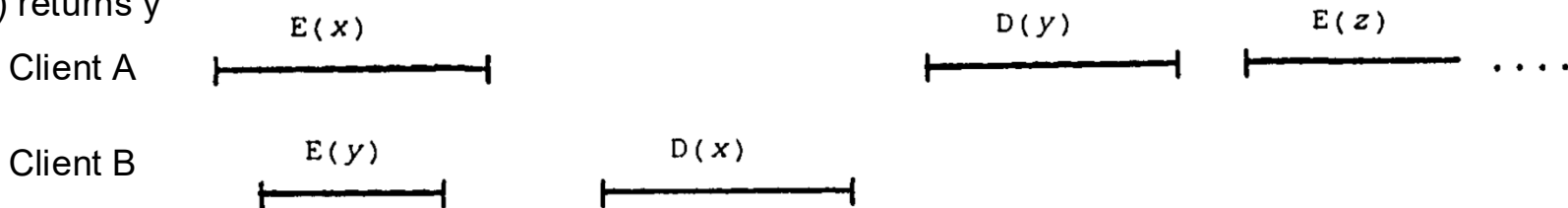
The key point is: whether the application agrees that the (potentially different) order(s) the shards/replicas see is still deemed as “correct”

The correct order – Linearizability

What is the meaning of “real-time order” in linearizability if there are concurrent operations?

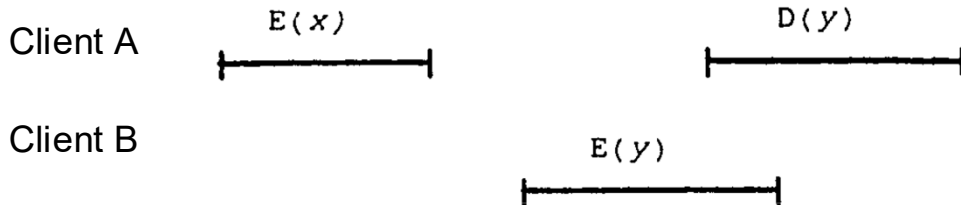
E(x): Enqueue(x)

D(y): Dequeue() returns y



(a) H_1 (acceptable).

The concurrent enqueues ends up like $E(x)$ then $E(y)$



$E(x)$ is clearly enqueued before $E(y)$
So, it is intuitively incorrect to see the first dequeue returns y

(b) H_2 (not acceptable).

The correct order – Linearizability

Behave like 1 sequential machine with

- All operations, including the concurrent ones,
- after *linearizing* (flattening) them onto the real timeline,
- form a *legal* (i.e., *reasonable based on the target object, e.g., queue*) linear order that honors the original history
- Applicable to both the shared-memory and message-passing models

Linearizability: A Correctness Condition for Concurrent Objects

MAURICE P. HERLIHY and JEANNETTE M. WING
Carnegie Mellon University

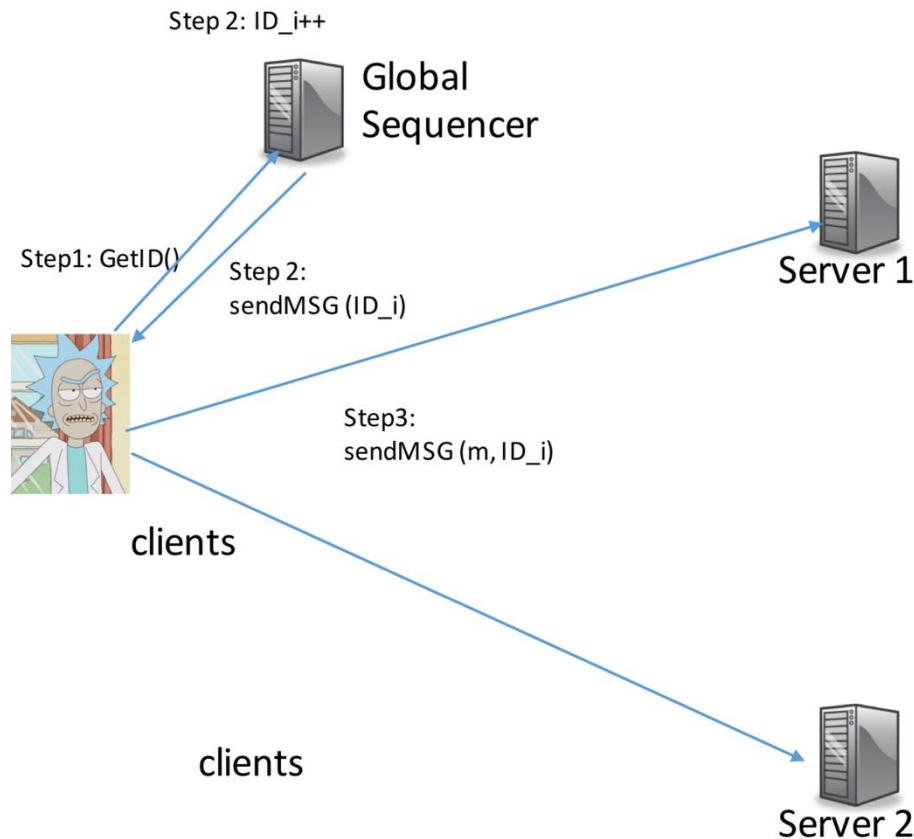
A concurrent object is a data object shared by concurrent processes. Linearizability is a correctness condition for concurrent objects that exploits the semantics of abstract data types. It permits a high degree of concurrency yet it permits programmers to specify and reason about concurrent objects

The gap between what you want and what you can achieve

- Linearizable in Distributed systems?
 - FLP – asynchronous setting (worst case)

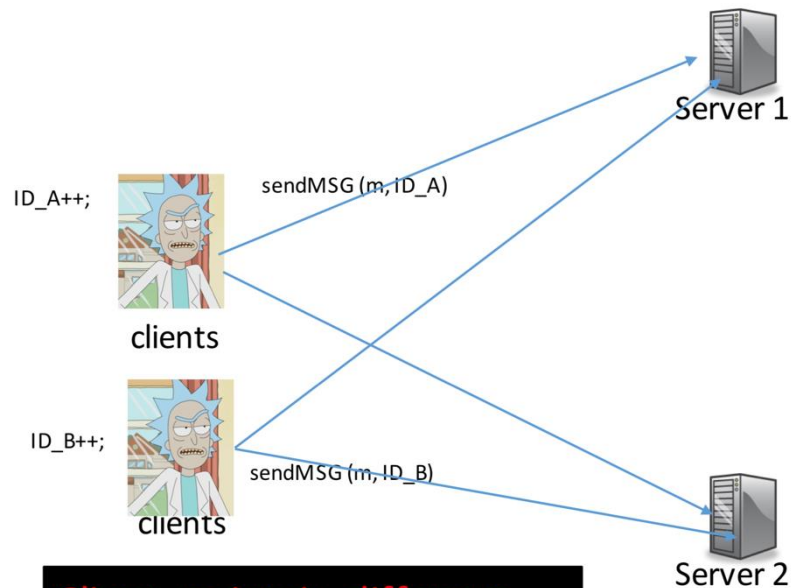
Implementing total-order broadcast

- Need a global sequencer (or elect a leader first)
- Before sending messages to all servers, must ask a sequencer for a timestamp
- No reliability yet
 1. What if replica 1 has no response for 1 min?
 - Then, the problem “upgrades” to be an “**atomic** total-order broadcast” problem
 2. What if the leader dies?
 - Then, it has a leader election problem
- 1+2 = **Replication State Machine**



Implementing FIFO-order broadcast

- Each client maintains its local counter
- Server maintains a queue for each client to order its messages

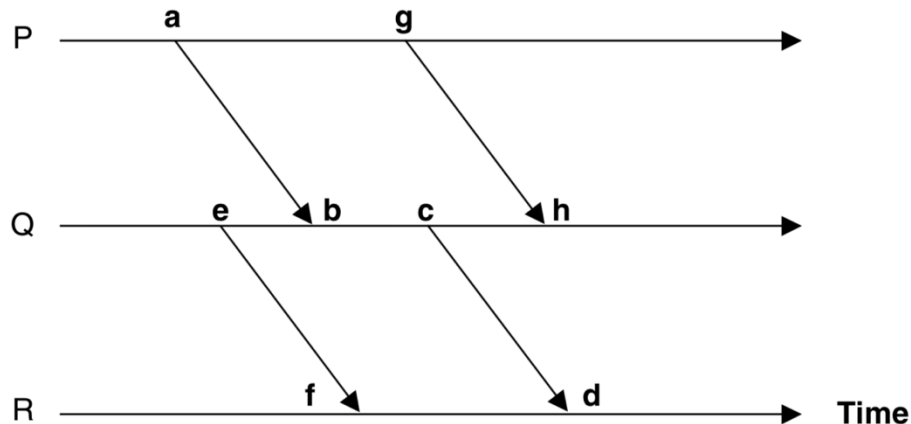


**Client maintain different
IDs: local and monotonic**

The classic: Happen-Before Ordering

- Definition:

- If same process
 - It is clear $a < g$
- If different processes P and Q
 - Event a: P sends Q a message
 - Event b: Q receives the message
 - $a < b$



Happen-Before Ordering

- Transitive Property:

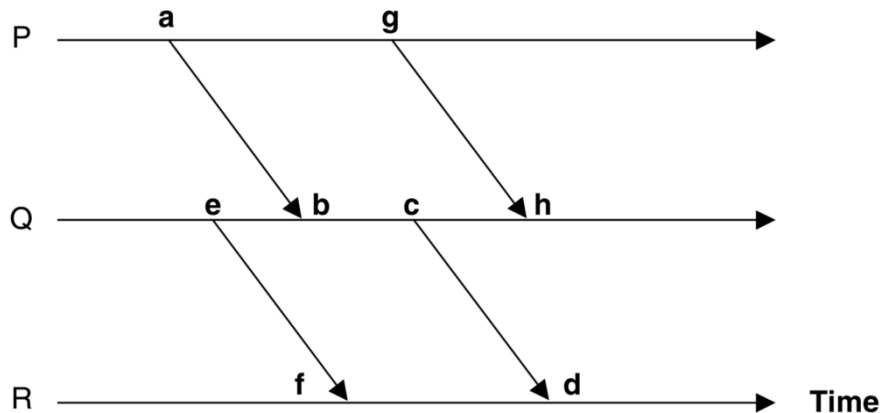
- If $(a < b)$ and $(b < c) \Rightarrow a < c$

- So, we have

- $e < d$
 - $a < d$

- Goal: Say, after the system finishes, if I ask P, Q, and R (not ask you),

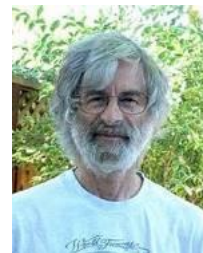
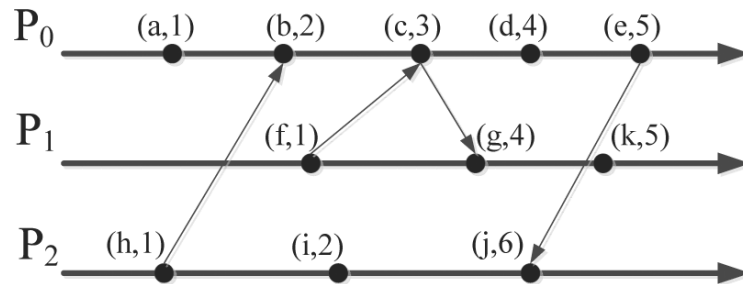
- “did b happen-before d?”
 - What kind of log info shall the processes keep to answer that question?



How to log a happen-before ordering among all events?

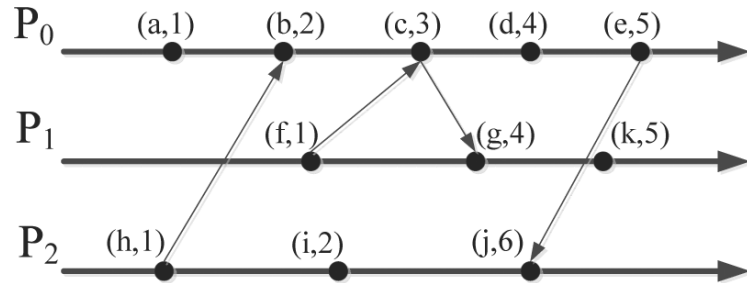
- Lamport's Logical Clock

- Clock is a.k.a. “timestamp”
- Each process has its own local counter
- 1) Any local things taken place, $LC++$
- 2) When sending out a message, piggyback its LC in the message
- 3) When receiving a message, set local $LC = 1 + \max(LC, \text{message's LC})$
 - i.e., make sure the recipient's time = 1 + time-when-sender-sends-its-message
 - **To make sure that the message is “arrived” *after* the sender's sent it**
- By looking at the clock,
 - We see that logical clock yields a partial order among events
 - E.g., a, h, f are all '1'
 - If somehow you need to ensure $P_0 P_1 P_2$ share the exact total-order
 - Just don't break the ties randomly, but break the tie of a, h, f, say, by adding the process id to the timestamp



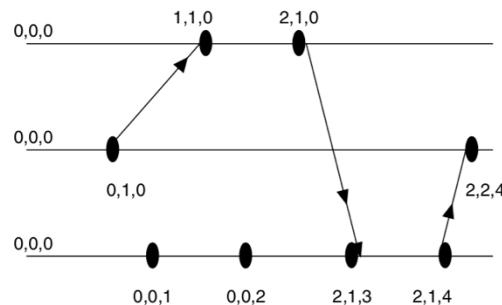
Limitations of Logical Clock

- So, if $x < y \implies LC(x) < LC(y)$
- But: $LC(x) < LC(y) \not\Rightarrow x < y$
 - E.g., $(g,4) < (j,6) \neq g < j$ by the definition of happen-before-ordering
- **Logical clock can answer: if $LC(x) < LC(y) \Rightarrow$**
 - x may (i) happen-before y, (ii) **or** concurrent with y,
 - Corollary of (i) and (ii): x surely did not happen-after y
 - For many applications, we don't need to differentiate (i) and (ii). E.g., $(d,4)$, $(g,4)$, and $(e,5)$
 - For mutex, both $d \rightarrow g \rightarrow e$ and $g \rightarrow d \rightarrow e$ are fine (as long as no starvation)
- That is, logical clock (happen-before) cannot tell **causality**, but just **potential causality**
- How to design a clock that can tell?



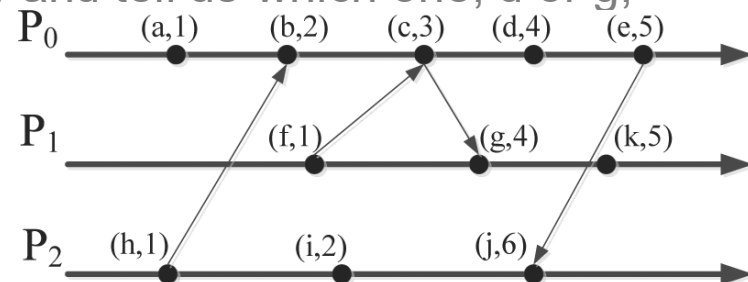
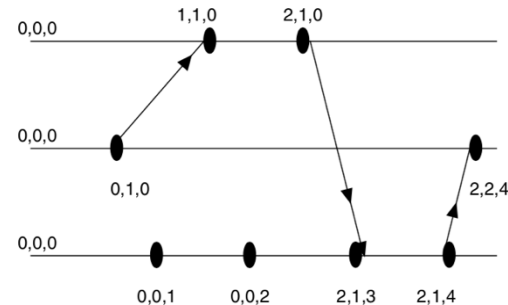
How to log a casual ordering among events?

- Vector Clock
- If $a < b \iff VC(a) < VC(b)$
- VC is not a value, but a vector of length N, if there are N processes
 - Process i: $VC(T_1^i, \dots, T_i^i, T_j^i, \dots, T_n^i)$
 - $VC(a) < VC(b)$ [based on dominance (prevail)]
- VC implementation:
 - 1) Any local things taken place at process i: $VC[i]++$
 - 2) When process i sends out a message, piggyback its VC in the message
 - 3) When process j receives a message from process i,
 - set j's VC from $(T_1^j, \dots, T_i^j, T_j^j, \dots, T_n^j)$ to $(\max(T_1^i, T_1^j), \dots, \max(T_i^i, T_i^j), \dots, \max(T_n^i, T_n^j))$



Vector Clock

- $a < b \iff VC(a) < VC(b)$
- Can tell whether it is concurrent or happen-before
 - Consider events $(0,0,1)$ and $(2,1,0)$
 - Means they are concurrent $||$
- If the events are more clearly stated as “read” and “write”
 - Then, $write(x) < read(x) < write(x)$
 - Can precisely capture causal relationship
- Exercise: work out the vector clock for the below and tell us which one, d or g, cause e?



Summary of ordering

- Linearizability: all replicas process the same linearizable order
 - Parallel systems: use of locks in concurrent data structures
 - Parallel systems = Synchronous system ==? Not subject to FLP
 - (Asynchronous) Distributed systems? Impossible in theory
- Total order: all replicas process the **same** sequential order of events
 - Require a global sequencer or elect a leader first
- FIFO ordering: all replicas honor each client's local order
 - Way easier to implement: each client's local clock is monotonic
 - Only useful if clients touch different things, else it doesn't work because
 - it can't capture the order between clients on the same item
- Causal ordering
 - Achievable by vector clock
 - But vector clock's message size increases with # of processes
- Happen-before ordering
 - Ex application: mutex

Summary of clock (a.k.a. timestamp, tag, etc)

- Designed to help establish the desired level of ordering in a cloud system

